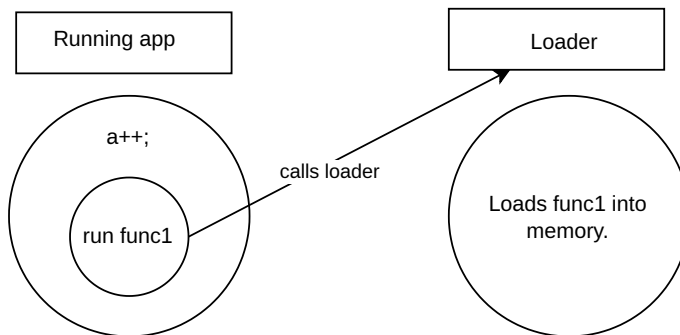


For linux usage, this is under GPL. For other systems, you must contact me.
andrealbergaria@gmail.com

Load/demand using functions

Everything should be loaded (into RAM), only when it's required. If an event (generally speaking) occurs, you can load more data (if needed). For example, we mount only the USB drives (or any other filesystem), only when we want. They are not all of them, mounted. This is a good example, of loading only stuff when we need. On mounting, user is responsible, for loading. Should stuff be loaded automatically?

Diagram 1

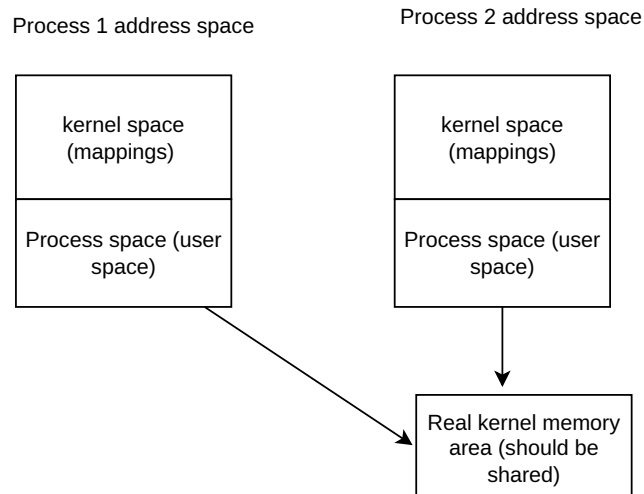


On Diagram 1, we use c syntax (a++), and natural language as an example...

For the loader, you need a protocol to specify
-what to load
-how

Process address space and kernel space

Diagram 2



On process 1 and 2, the kernel space, is part of process address space. Why do we call it kernel space? Usually kernel space, is where you load / execute instructions, that are used / belong to kernel. Also , kernel space, is different either we use process addresses kernel space, or use the actual kernel reserved memory. If you access the kernel space on process address space, you're accessing both process memory, and kernel memory. So on processes instead of using kernel space, use pointers to actual kernel memory? Not using kernel space on process, frees a lot of address space...

Why use kernel mappings on process address space?

According to chat GPT,
"This design avoids changing page tables on every system call or interrupt, which was historically expensive because it required flushing the TLB."

With this system (load on demand), possibly, there won't be too costly, to put / take pages into / , out of memory

So process 1, calls int 0x80. Will it run code on process address space? It's ambiguous..it runs on process address space, and also on kernel memory? Can int 0x80 use actual reserved kernel memory? Different from accessing from the process address space? Shouldn't that be supposed to happen?

RAM needed

How much RAM we need, per process (or by other construct), to run an app, using demand / load, as in , load only functions that are called? At one time, how much data we have present at RAM ? (using function demand/load)? 10kb ? 30kb?

Depends, on function? , let the user choose? Again performance issues. On loader, can we set buffer limits? or just load part of buffer, or load any function we want.

Boot

Why load a lot of services, at boot, that aren't used? Just load on demand. Still needs to load some dependencies, that are required by basic services.

Impact

If, we have 30kb of process address space, can we increase the address space by ways of kernel, or others? We just need to stop process, create a new process with different address space.

There might be performance issues...less memory used though.

Will interrupts be easier / faster with load / demand functions?

Protected mode, already, does, if data is not on memory, raises a page fault. (that means to load page into memory). Can we access protected mode (the system, not just protected mode), to check which function is loaded, and what is its address?

Logistics problem (related to performance, and stalls or crashes), Many functions inside function calls? Just wait for all, or just some, to return.

Also, if you use load on demand, it's easier to store the previous state, and stop / resume it , since we are running one function at a time...so. new process context switch?

Since we are running one function at a time, no process context switch.

Better multitasking ? Faster switching
processes?